

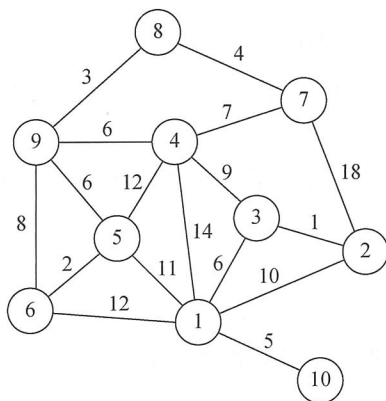
提高组 CSP-S 2024 初赛模拟卷 9

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 p 进制下 ($9 \leq p \leq 16$) 有如下等式成立: $7*8=44$, 则该进制下 $52*74=$ ()。
A. 2b88 B. 6365 C. 3848 D. 28b5
2. 在 8 位二进制补码中, 10101101 表示的数是十进制下的 ()。
A. 45 B. -45 C. -83 D. 173
3. 后缀表达式 $9 \ 3 \ 1 \ - \ 3 \ * \ + \ 10 \ 2 \ / \ +$ 的计算结果为 ()。
A. 20 B. 1030 C. 2 D. 15
4. 7 个同学坐在一排, 由于现在要考试, 为了避免同学们在自己的桌子上留下小抄作弊, 老师打算打乱所有人的位置, 以保证每个人都不坐在自己原来的位置, 那么一共有 () 种打乱方式。
A. 265 B. 1854 C. 720 D. 1440
5. 若某算法的计算时间表示为递推关系
$$T(n) = 2T(n/2) + n \log_2 n$$
则该算法的时间复杂度为 ()。
A. $O(n \log^3_2 n)$ B. $O(n \log^2_2 n)$ C. $O(n \log_2 n)$ D. $O(n)$
6. 若让元素 1, 2, 3, 4, 5 依次进栈, 则出栈次序不可能出现 () 的情况。
A. 5, 4, 3, 2, 1 B. 2, 1, 5, 4, 3 C. 4, 3, 1, 2, 5 D. 1, 2, 5, 4, 3
7. NOI Linux 2 是 CCF NOI 系列赛事的标准竞赛环境之一。以下终端命令中, 能够显示当前目录名称的是 ()。
A. cat B. ls C. man D. pwd
8. 以下程序段的作用是求出 1 至 n 的所有 ()。
A. 素数个数 B. 整数的最大因数
C. 整数的因数个数 D. 整数的 $\phi(i)$

```
for (i=1;i<=n;i++) a[i]=i;
for (i=2;i<=n;i++)
    if (i==a[i])
    {
        for (j=1;j*i<=n;j++)
            a[j*i]=a[j*i]/i*(i-1);
    }
```

9. 给定地址区间为 0~12 的哈希表, 哈希函数为 $h(x) = x^2 \% 13$, 采用线性探查的冲突解决策略 (若出现冲突情况, 会往后探查第一个空的地址存储; 若地址 12 冲突了, 则从地址 0 重新开始探查)。哈希表初始为空表, 依次存储(3, 4, 5, 6, 7, 8, 9, 10)后, 10 存储在哈希表的哪个地址中? ()
- A. 0 B. 1 C. 9 D. 10
10. 令根结点的高度为 1, 则一棵含有 2023 个结点的二叉树和一棵含有 2023 个结点的三叉树的高度分别至少为 ()。
- A. 11, 8 B. 11, 7 C. 10, 7 D. 10, 8
11. 以下选项中哪一个的期望时间复杂度与其他三个不同? ()
- A. 包含 n 个元素的 set 容器使用 lower_bound 查询返回第一个大于某个数的迭代器
- B. 包含 n 个元素的 vector 使用 erase 删除 $n/2$ 位置的值
- C. 从包含 n 个元素的数组中构建一个小顶堆
- D. 求包含 n 个结点的树的直径
12. 该无向图的最小生成树的权值为 ()。



- A. 39 B. 38 C. 47 D. 42

13. 以下说法中不正确的是 ()。
- A. 无向图具有欧拉回路的充分必要条件是每个点的度数均为奇数
 - B. 在稠密图上, 相较于堆优化的 Dijkstra 算法, SPFA 表现更优
 - C. 无向图是二分图, 当且仅当图上不存在奇数长度的环
 - D. 树的重心不一定唯一
14. 掷 7 次质地均匀的正方体骰子, 出现和为 11 的概率可表示为 $\frac{x}{6^7}$, 则 x 的值为 ()。
- A. 84 B. 210 C. 121 D. 240
15. 设循环队列中数组的下标范围是 $1 \sim m$, 其头尾指针分别为 $front$ 和 $rear$, $front$ 指向队列的第一个元素, $rear$ 指向最后一个元素位置的下一个位置, 则其元素个数为 ()。
- A. $rear - front$ B. $rear - front + 1$
C. $(rear - front + 1) \text{ MOD } m$ D. $(rear - front + m) \text{ MOD } m$

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(1)

```
01 #include<iostream>
02 #include<algorithm>
03 #define Type double
04 using namespace std;
05
06 const int maxn = 100005;
07 struct point
08 {
09     Type x, y;
10     point() {}
11     point( Type x, Type y ) : x(x), y(y) {}
12     point operator - ( const point b ) const
13     {
14         return point( x - b.x, y - b.y );
15     }
16     void input()
17     {
18         cin >> x >> y;
19     }
```

```
20 } pnt[maxn], res[maxn];
21
22 Type cross( point a, point b )
23 {
24     return a.x * b.y - a.y * b.x;
25 }
26
27 int solve( int n )
28 {
29     sort( pnt, pnt + n, [&](point a, point b)
30     {
31         return a.x==b.x? a.y<b.y : a.x<b.x;
32     } );
33     int m = 0;
34     for( int i = 0; i < n; ++i )
35     {
36         while( m > 1 && cross( res[m-1] - res[m-2],
37                               pnt[i] - res[m-1] ) <= 0 )
38             --m;
39         res[m++] = pnt[i];
40     }
41     int k = m;
42     for( int i = n - 2; i >= 0; --i )
43     {
44         while( m > k && cross( res[m-1] - res[m-2],
45                               pnt[i] - res[m-1] ) <= 0 )
46             --m;
47         res[m++] = pnt[i];
48     }
49     if( n > 1 ) --m;
50     return m;
51 }
52 int main()
53 {
54     int n;
55     cin>>n;
56     for(int i=0; i<n; ++i) pnt[i].input();
57     cout << solve(n) << endl;
58 }
```

假设输入的所有数均为整数，且绝对值都不超过 100 000，以下是 3 个输入样例。

输入样例 1:	输入样例 2:	输入样例 3:
5	11	6
0 0	0 4	0 0
0 2	2 8	4 4
2 2	4 4	0 4
2 1	4 1	2 2
2 0	8 2	2 0
	8 8	4 0
	9 6	
	14 0	
	12 4	
	13 8	
	12 10	

■ 判断题

16. 在输入样例 1 的情况下，结果为 5。()
17. 将第 3 行中的 double 修改为 int，所有合法输入的结果均不会发生变化。()
18. 将第 36 行和第 43 行中的 <= 均修改为 <，输入样例 1 的输出结果不会发生变化。()
19. 该算法的时间复杂度为 $O(n\log n)$ 。()

■ 选择题

20. 在输入样例 2 的情况下，结果为 ()。
A. 4 B. 5 C. 6 D. 7
21. (4 分) 在输入样例 3 的情况下，第 37 行和第 44 行的运行次数加起来一共为 ()。
A. 5 B. 6 C. 7 D. 8

(2)

```
01 #include<iostream>
02 #include<algorithm>
03 using namespace std;
04
```

```
05 const int maxn = 2e5+7;
06
07 int func1(const char str[], int n)
08 {
09     int k=0,i=0,j=1;
10     for(; j<n; j++)
11     {
12         while(k<n&&str[i+k]==str[j+k])
13         {
14             k++;
15         }
16         if(str[i+k]>str[j+k])
17             i=j;
18         k=0;
19     }
20     return i;
21 }
22
23 int func2(const char str[],int n)
24 {
25
26     int i=0,j=1,k=0;
27     while(i<n && j<n && k<n)
28     {
29         if(str[i+k]==str[j+k])
30             k++;
31         else
32         {
33             str[i+k] > str[j+k] ? i+=k+1 : j+=k+1;
34             if(i==j)
35                 i++;
36             k=0;
37         }
38     }
39     return min(i,j);
40 }
41
42 char str[maxn];
43
44 int main()
45 {
```

```
46     int n;  
47     cin >> n >> str;  
48     for(int i=0;i<n;++i) str[i+n] = str[i];  
49     cout << func1(str, n) << ' ' << func2(str, n) << endl;
```

假设输入均为整数加一个字符串，其中整数表示字符串的长度，整数不超过 100 000。

■ 判断题

22. 若输入 5 12312，则输出的第一个整数为 4。 ()
23. 将第 12 行中的 `while(k<n&&str[i+k]==str[j+k])` 修改为 `while(str[i+k]==str[j+k])`，则程序有可能会陷入死循环或出现数组越界。 ()
24. 将第 35 行中的 `i++` 修改为 `j++`，结果不会发生变化。 ()
25. 在合法输入下，输出的两个整数一定是一样的。 ()

■ 选择题

26. (4 分) 输入 9 321321323 时，第 33 行一共运行了 () 次。
A. 8 B. 7 C. 6 D. 5
27. (4 分) 假设字符串 s 是一个长度为 3 且只包含数字 0 到 9 的字符串，其中 s 的每个位置为 0 到 9 的可能性相同，则对该程序输入 3 和字符串 s 时，输出的第二个整数是 0 的概率为 ()。
A. $\frac{1}{3}$ B. $\frac{37}{100}$ C. $\frac{17}{50}$ D. $\frac{2}{5}$

(3)

```
01 #include<iostream>  
02 #include<vector>  
03 using namespace std;  
04 const int maxn = 1e6+5;  
05  
06 vector<int> pri;  
07 bool not_prime[maxn];  
08 int value[maxn], num[maxn];  
09  
10 int init(int n)  
11 {  
12     value[1] = 1;
```



```
13     for (int i = 2; i <= n; ++i)
14     {
15         if (!not_prime[i])
16         {
17             pri.push_back(i);
18             value[i] = 2;
19             num[i] = 1;
20         }
21         for (int j : pri)
22         {
23             if (i * j > n) break;
24             not_prime[i * j] = true;
25             if (i % j == 0)
26             {
27                 num[i * j] = num[i] + 1;
28                 value[i * j] = value[i] / num[i * j] * (num[i * j] + 1);
29                 break;
30             }
31             num[i * j] = 1;
32             value[i * j] = value[i] * 2;
33         }
34     }
35     return value[n];
36 }
37
38 int main()
39 {
40     int n;
41     cin >> n;
42     cout << init(n) << endl;
43 }
```

假设输入为一个不超过 1 000 000 的整数。

■ 判断题

28. 输入 180 时, 输出结果为 18。 ()
29. 分别输入 180 和 300 时, 在 main 函数退出前, 数组 num 在下标 1 到 100 上的值是不同的。 ()
30. 输入 100 时, 在 main 函数退出前, vector 容器 pri 内的元素个数在 22 和 28 之间。 ()

31. 将第 28 行修改为 $\text{value}[i * j] = \text{value}[i] * (\text{num}[i * j] + 1) / \text{num}[i * j]$, 结果不变。 ()

■ 选择题

32. 该算法的时间复杂度为 ()。
- A. $O(n)$ B. $O(n \log n)$ C. $O(n \log \log n)$ D. $O(\sqrt{n})$
33. (4 分) 数组 value 在下标 i 处的含义为 ()。
- A. i 的质因子个数 B. i 的欧拉函数值
C. i 的所有因子和 D. i 的因子个数

三、完善程序 (单选题, 每小题 3 分, 共计 30 分)

- (1) 有 N 种物品和一个容量为 V 的背包。

物品一共有 3 类:

第一类物品只能用 1 次 (0-1 背包);

第二类物品可以用无限次 (完全背包);

第三类物品最多只能用 s_i 次 (多重背包)。

每种物品的体积是 v_i , 价值是 w_i 。求解将哪些物品装入背包, 可使物品体积总和不超过背包容量, 且价值总和最大。输出最大价值。

输入格式:

第 1 行有两个整数 N, V , 用空格隔开, 分别表示物品种数和背包容量。接下来有 N 行, 每行 3 个整数 v_i, w_i, s_i , 用空格隔开, 分别表示第 i 种物品的体积、价值和数量。

$s_i = -1$ 表示第 i 种物品只能用 1 次;

$s_i = 0$ 表示第 i 种物品可以用无限次;

$s_i > 0$ 表示第 i 种物品可以用 s_i 次。

输出格式:

输出一个整数, 表示最大价值。

数据范围:

$$0 < N, V \leq 1000$$

$$0 < v_i, w_i \leq 1000$$

$$-1 < s_i \leq 1000$$

输入样例:

```
4 5
1 2 -1
2 4 1
3 4 0
4 5 2
```

输出样例:

```
8
```

提示: 解决思路是将多重背包使用二进制拆分转换为 0-1 背包, 使得只包含 0-1 背包和完全背包。

```
01 #include <iostream>
02 #include <algorithm>
03 #include <vector>
04 using namespace std;
05 const int maxn = 2023;
06
07 int N, V;
08 int dp[maxn];
09 struct Item
10 {
11     int w, v;
12     int s;
13 };
14 void upd(int j, Item item)
15 {
16     dp[j] = max(dp[j], ① );
17 }
18 int main()
19 {
20     vector<Item> items;
21     cin >> N >> V;
22
23     for (int i = 1; i <= N; i++)
24     {
25         int w, v, s;
26         cin >> w >> v >> s;
27         if ( ② )
```

```
28     {
29         for (int k = 1; k <= s; k <= 1)
30         {
31             s -= k;
32             items.push_back({ ③ });
33         }
34         if (s > 0)
35             items.push_back({s * w, s * v, -1});
36     }
37     else
38         items.push_back({w, v, s});
39 }
40 for (auto item : items)
41 {
42     if (item.s == ④ )
43         for (int j = V; j >= item.w; j--)
44             upd(j, item);
45     else
46         for ( ⑤ )
47             upd(j, item);
48 }
49 cout << dp[V] << endl;
50 return 0;
51 }
```

34. ①处应填 ()。

- | | |
|------------------------------|------------------------------|
| A. $dp[j - item.v] + item.w$ | B. $dp[j - item.w] + item.v$ |
| C. $dp[j + item.v] + item.w$ | D. $dp[j + item.w] + item.v$ |

35. ②处应填 ()。

- | | | | |
|------------|-------------|--------------|-------------|
| A. $s > 0$ | B. $s == 0$ | C. $s == -1$ | D. $s != 0$ |
|------------|-------------|--------------|-------------|

36. ③处应填 ()。

- | | |
|-----------------------|-------------------|
| A. $w * k, v * k, -1$ | B. $w * k, v, -1$ |
| C. $w, v, -1$ | D. $w * k, v, 0$ |

37. ④处应填 ()。

- | | | | |
|------|------|------|-------|
| A. 1 | B. N | C. 1 | D. -1 |
|------|------|------|-------|

38. ⑤处应填 ()。

- A. `int j = item.w; j <= V; j-`
- B. `int j = item.v; j <= V; j++`
- C. `int j = item.w; j <= V; j ++`
- D. `int j = V; j>=item.w ; j-`

(2) 给定 n 个正整数 $a_1, a_2, \dots, a_n$, m 次询问, 每次询问一个给定的区间 $[l, r]$, 输出 $a_l, a_{l+1}, \dots, a_r$ 的最大公因数。

输入格式:

第 1 行两个整数 n, m 。第 2 行 n 个整数表示 $a_1, a_2, \dots, a_n$ 。以下 m 行, 每行两个整数 l, r , 表示询问区间的左右端点。

输出格式:

共 m 行, 每行表示一个询问的答案。

数据范围:

$$1 \leq l \leq r \leq n \leq 100\,000$$

$$1 \leq m \leq 10^6$$

$$1 \leq a_i \leq 10^9$$

输入样例:

```
5 3
4 12 3 6 7
1 3
2 3
5 5
```

输出样例:

```
1
3
7
```

```
01 #include <iostream>
02 #include <algorithm>
03 using namespace std;
04 const int maxn = 2e5 + 7;
05 const int maxp = 20;
06
07 int a[maxn], dp[maxn][maxp];
```

```
08 int gcd(int a, int b)
09 {
10     if (b == 0)
11         return a;
12     ①;
13 }
14 void init(int n)
15 {
16     for (int i = 1; i <= n; i++)
17         dp[i][ ② ] = a[i];
18     for (int j = 1; (1 << j) <= n; j++)
19         for (int i = 1; i + (1 << j) - 1 <= n; i++)
20             dp[i][j] = gcd(dp[i][j - 1], dp[ ③ ][j - 1]);
21 }
22 int rmq(int l, int r)
23 {
24     int k = 0;
25     while ( ④ )
26         k++;
27     return gcd(dp[l][k], dp[ ⑤ ][k]);
28 }
29 int main()
30 {
31     int n, m;
32     cin >> n >> m;
33     for (int i = 1; i <= n; i++)
34         cin >> a[i];
35     init(n);
36     for (int i = 1; i <= m; i++)
37     {
38         int l, r;
39         cin >> l >> r;
40         cout << rmq(l, r) << endl;
41     }
42 }
```

39. ①处应填 ()。

- A. gcd(b, a%b) B. gcd(b, a*b) C. gcd(a, a%b) D. gcd(a, a/b)

40. ②处应填 ()。

A. 1

B. 0

C. -1

D. i

41. ③处应填 ()。

A. $i+(1 \leq (j-1))-1$

B. $i+(1 \leq j)-1$

C. $i+(1 \leq j)$

D. $i+(1 \leq (j-1))$

42. ④处应填 ()。

A. $(1 \leq k) \leq r-l+1$

B. $(1 \leq (k+1)) \leq r-l+1$

C. $(k \leq 1) \leq r-l+1$

D. $((k+1) \leq 1) \leq r-l+1$

43. ⑤处应填 ()。

A. $r-(1 \leq k)+1$

B. $r-(1 \leq k)$

C. $1+(1 \leq k)-1$

D. $1+(1 \leq k)$